

✓ Lab 05: RNNs vs. Transformers vs. Vision Transformers

ITAI 2376 — Deep Learning | Spring 2026

Houston City College — AI & Robotics Program

Welcome to Lab 05! This is a comprehensive, hands-on lab where you will build, train, and compare three major deep learning architectures:

Part	Architecture	Task	Dataset
A	LSTM & GRU (RNNs)	News Topic Classification	AG News
B	DistilBERT (Transformer)	News Topic Classification	AG News (same data!)
C	Vision Transformer (ViT)	Image Classification	CIFAR-10
D	—	Comparative Analysis & Reflection	All results

Why this lab matters: By using the *same text dataset* for both RNNs and Transformers, you get a direct, apples-to-apples comparison. Then, by exploring Vision Transformers, you see how the Transformer idea extends beyond text into images — and you can compare ViT performance to the CNNs you built in Module 03.

Estimated Time: 3–4 hours

Environment: Google Colab (free tier is sufficient)

⚠ **Important:** Run cells in order. If you restart the runtime, re-run all cells from the top.

✓ Part 0: Environment Setup

First, let's install libraries and check GPU access. GPU is not strictly required but makes training much faster.

If no GPU: Runtime → Change runtime type → T4 GPU. If unavailable, reduce epochs or dataset size as noted in the troubleshooting section at the end.

```
# Cell 0.1 – Install Required Libraries
# This cell installs the external libraries needed for this lab:
#   - transformers: Hugging Face library for pre-trained models (BERT, ViT)
#   - datasets: Hugging Face library for loading standard datasets (AG News, CI
#   - accelerate: Helps optimize training on different hardware (GPU/CPU)
# The -q flag makes installation quiet (less verbose output)
```

```
!pip install transformers datasets accelerate -q
print("✅ Libraries installed!")
```

✅ Libraries installed!

Cell 0.2 – Import Core Libraries and Check GPU Availability

```
# ===== DEEP LEARNING FRAMEWORK =====
import torch                    # PyTorch: main deep learning framework
import torch.nn as nn          # Neural network modules (layers, loss fu
import torch.optim as optim    # Optimization algorithms (Adam, SGD, etc
from torch.utils.data import DataLoader, Dataset # Data loading utilities for

# ===== DATA PROCESSING =====
import numpy as np             # Numerical operations on arrays
from collections import Counter # Count word frequencies for vocabulary b
import re                      # Regular expressions for text cleaning

# ===== VISUALIZATION =====
import matplotlib.pyplot as plt # Plotting library for charts and graphs

# ===== UTILITIES =====
import time                    # Track training time per epoch
import warnings                # Suppress non-critical warnings
warnings.filterwarnings('ignore') # Hide deprecation and other warnings for

# ===== HUGGING FACE LIBRARIES (imported later when needed) =====
# from datasets import load_dataset # For loading AG News dataset
# from transformers import ...      # For DistilBERT and ViT models
# from torchvision import ...       # For CIFAR-10 dataset and image transfor

# ===== DEVICE CONFIGURATION =====
# Check if CUDA-enabled GPU is available; otherwise use CPU
# Training on GPU is 10-50x faster than CPU for deep learning
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"💻 Device: {device}")

if device.type == 'cuda':
    # Display GPU name (e.g., Tesla T4, A100)
    print(f"    GPU: {torch.cuda.get_device_name(0)}")
else:
    print(f"    ⚠️ No GPU detected. Training will be slower. See troubleshootin
```

```
💻 Device: cuda
    GPU: Tesla T4
```

✓ Part A: RNN Text Classification (LSTM & GRU)

In Module 04, you learned about Recurrent Neural Networks — architectures that process sequences step by step, maintaining a hidden state that acts as "memory." Now you put that knowledge into practice.

Your task: Build LSTM and GRU classifiers to categorize news articles into four topics:

- 🌍 **World** — international news
- 🏈 **Sports** — athletic events
- 💼 **Business** — financial and corporate news
- 🔬 **Sci/Tech** — science and technology

We use the **AG News** dataset — a well-known benchmark. We use a subset to keep training fast on free Colab.

✓ Step A.1 — Load and Explore AG News

```
# Cell A.1 — Load AG News Dataset
# Import the Hugging Face datasets library for easy access to standard datasets
from datasets import load_dataset

# Load the AG News dataset (4-class news topic classification)
# This dataset contains news articles labeled as: World, Sports, Business, Sci/
dataset = load_dataset("ag_news")

# Define subset sizes to keep training time reasonable on free Colab
TRAIN_SIZE = 8000 # 2000 samples per class (4 classes × 2000 = 8000)
TEST_SIZE = 2000 # 500 samples per class (4 classes × 500 = 2000)

# Shuffle and select subsets (seed=42 ensures reproducibility)
train_data = dataset['train'].shuffle(seed=42).select(range(TRAIN_SIZE))
test_data = dataset['test'].shuffle(seed=42).select(range(TEST_SIZE))

# Define human-readable class names (labels are 0, 1, 2, 3)
CLASS_NAMES = ['World', 'Sports', 'Business', 'Sci/Tech']

# Display dataset information
print(f"Training samples: {len(train_data)}")
print(f"Test samples: {len(test_data)}")
print(f"\n--- Sample article ---")
print(f"Label: {CLASS_NAMES[train_data[0]['label']]}")
print(f"Text: {train_data[0]['text'][:300]}...")
```

```
README.md:      8.07k/? [00:00<00:00, 401kB/s]

data/train-00000-of-      18.6M/18.6M [00:04<00:00, 19.8MB/s]
00001.parquet: 100%

data/test-00000-of-      1.23M/1.23M [00:02<00:00, 453kB/s]
00001.parquet: 100%

Generating train split: 100%      120000/120000 [00:00<00:00, 432249.18 examples/s]

Generating test split: 100%      7600/7600 [00:00<00:00, 219950.12 examples/s]

Training samples: 8000
Test samples:      2000

--- Sample article ---
```

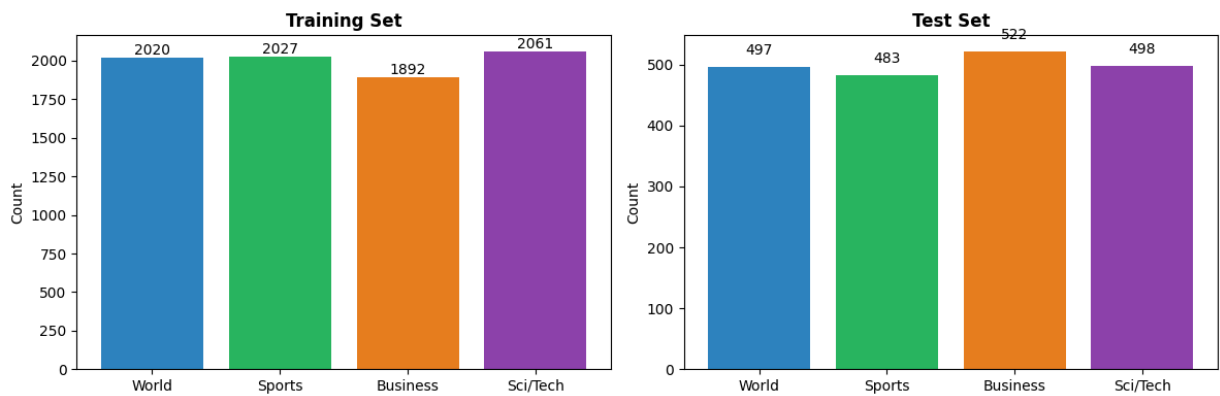
```
# Cell A.2 – Visualize Class Distribution
# It's important to check if classes are balanced (equal representation)
# Imbalanced datasets can bias the model toward majority classes

# Extract labels from training and test sets
train_labels = [s['label'] for s in train_data]
test_labels = [s['label'] for s in test_data]

# Create side-by-side bar charts
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

for ax, labels, title in zip(axes, [train_labels, test_labels], ['Training Set'
    # Count occurrences of each class
    counts = Counter(labels)
    # Create bar chart with distinct colors for each class
    bars = ax.bar(CLASS_NAMES, [counts[i] for i in range(4)],
        color=['#2E86C1', '#28B463', '#E67E22', '#8E44AD'])
    ax.set_title(title, fontweight='bold')
    ax.set_ylabel('Count')
    # Add count labels on top of each bar
    for bar, c in zip(bars, [counts[i] for i in range(4)]):
        ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+20,
            str(c), ha='center')

plt.tight_layout()
plt.show()
```



✓ Step A.2 — Text Preprocessing

Neural networks cannot process raw text. We need to:

1. **Tokenize** — split text into words
2. **Build a vocabulary** — assign a unique number to each word
3. **Convert to sequences** — replace words with numbers
4. **Pad sequences** — make all sequences the same length for batching

This is the same pipeline from Module 04, implemented from scratch.

Cell A.3 — Build Vocabulary and Preprocess Text

```
def tokenize(text):
    """
    Convert text to lowercase and split into words.

    Steps:
    1. Convert to lowercase ("Hello" and "hello" should be the same word)
    2. Remove punctuation and special characters (keep only letters, numbers, s
    3. Split on whitespace to get individual words

    Example: "Hello, World!" → ["hello", "world"]
    """
    text = text.lower()                # Convert to lowercase
    text = re.sub(r'^a-z0-9\s', '', text) # Remove punctuation
    return text.split()                # Split into words
```

```

# Count word frequencies across all training documents
# This helps us identify the most common words to include in our vocabulary
word_counts = Counter()
for sample in train_data:
    word_counts.update(tokenize(sample['text']))

# ===== VOCABULARY PARAMETERS =====
VOCAB_SIZE = 10000 # Keep only the 10,000 most common words
MAX_LEN = 128      # Truncate/pad all sequences to 128 tokens
PAD_IDX = 0        # Index for padding token (fills shorter sequences)
UNK_IDX = 1        # Index for unknown words (not in vocabulary)

# Select the top (VOCAB_SIZE - 2) most common words
# We reserve indices 0 and 1 for <PAD> and <UNK>
most_common = word_counts.most_common(VOCAB_SIZE - 2)

# Create word-to-index mapping (dictionary)
# Each word gets a unique integer ID starting from 2
word2idx = {word: idx+2 for idx, (word, _) in enumerate(most_common)}
word2idx['<PAD>'] = PAD_IDX # Special token for padding
word2idx['<UNK>'] = UNK_IDX # Special token for unknown words

print(f"Vocabulary size: {len(word2idx):,}")
print(f"Max sequence length: {MAX_LEN}")
print(f"Top 10 words: {[w for w, _ in most_common[:10]]}")

```

```

Vocabulary size: 10,000
Max sequence length: 128
Top 10 words: ['the', 'to', 'a', 'of', 'in', 'and', 'on', 'for', '39s', 'that']

```

```

# Cell A.4 – Create PyTorch Dataset
# PyTorch requires data in a specific format for efficient batching and loading

class NewsDataset(Dataset):
    """
    Custom PyTorch Dataset for AG News.

    Converts raw text into:
    - Padded sequences of word indices (input for the model)
    - Integer labels (target for classification)
    """
    def __init__(self, data, word2idx, max_len):
        self.texts, self.labels = [], []

        for sample in data:
            # Step 1: Tokenize text and convert words to indices
            # If a word is not in vocabulary, use UNK_IDX
            indices = [word2idx.get(w, UNK_IDX) for w in tokenize(sample['text'])

            # Step 2: Pad sequence to max_len (add PAD_IDX to the end)

```

```

        # This ensures all sequences have the same length for batching
        indices += [PAD_IDX] * (max_len - len(indices))

        # Step 3: Convert to PyTorch tensor
        self.texts.append(torch.tensor(indices, dtype=torch.long))
        self.labels.append(sample['label'])

    # Convert labels to tensor
    self.labels = torch.tensor(self.labels, dtype=torch.long)

    def __len__(self):
        """Return the number of samples in the dataset."""
        return len(self.labels)

    def __getitem__(self, idx):
        """Return a single sample (text, label) at the given index."""
        return self.texts[idx], self.labels[idx]

# ===== CREATE DATASETS AND DATA LOADERS =====
BATCH_SIZE = 64 # Process 64 samples at a time (adjust based on GPU memory)

# Create dataset objects
train_dataset = NewsDataset(train_data, word2idx, MAX_LEN)
test_dataset = NewsDataset(test_data, word2idx, MAX_LEN)

# Create data loaders (handle batching and shuffling)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE)

# Display sample information
sample_text, sample_label = train_dataset[0]
print(f"Text tensor shape: {sample_text.shape}")
print(f"Label: {sample_label.item()} ({CLASS_NAMES[sample_label.item()]})")
print(f"Training batches: {len(train_loader)}")

Text tensor shape: torch.Size([128])
Label: 0 (World)
Training batches: 125

```

✓ Step A.3 — Build the LSTM Model

Architecture:

1. **Embedding layer** — converts word indices into dense vectors
2. **Bidirectional LSTM** — reads the sequence in both directions
3. **Fully connected layer** — maps LSTM output to 4 class predictions

```
# Cell A.5 — LSTM Classifier
```

```

class LSTMClassifier(nn.Module):
    """
    LSTM-based text classifier.

    Architecture:
    1. Embedding: Converts word indices to dense vectors (e.g., 5234 → [0.2, -0.1, ...])
    2. Bidirectional LSTM: Processes sequence forward and backward to capture context
    3. Dropout: Prevents overfitting by randomly dropping connections during training
    4. Fully Connected: Maps LSTM output to class probabilities
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes, dropout=0.3):
        super().__init__()

        # Embedding layer: vocab_size × embed_dim matrix
        # Each word index is mapped to a dense vector of size embed_dim
        # padding_idx=PAD_IDX ensures padding tokens don't contribute to learning
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_IDX)

        # Bidirectional LSTM layer
        # - embed_dim: size of input vectors (from embedding)
        # - hidden_dim: size of hidden state (LSTM memory)
        # - batch_first=True: input shape is (batch, sequence, features)
        # - bidirectional=True: process sequence left-to-right AND right-to-left
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)

        # Dropout layer: randomly sets 30% of activations to zero during training
        # This prevents the model from relying too heavily on specific features
        self.dropout = nn.Dropout(dropout)

        # Fully connected layer: maps LSTM output to class scores
        # hidden_dim * 2 because bidirectional LSTM concatenates forward and backward states
        self.fc = nn.Linear(hidden_dim * 2, num_classes)

    def forward(self, x):
        """
        Forward pass through the network.

        Args:
            x: Input tensor of word indices, shape (batch_size, max_len)

        Returns:
            Class scores, shape (batch_size, num_classes)
        """
        # Step 1: Convert word indices to embeddings
        # Shape: (batch_size, max_len) → (batch_size, max_len, embed_dim)
        embedded = self.embedding(x)

        # Step 2: Pass through LSTM
        # output: all hidden states, shape (batch_size, max_len, hidden_dim * 2)
        # hidden: final hidden states, shape (2, batch_size, hidden_dim)
        # [0] = forward direction, [1] = backward direction
        output, hidden = self.lstm(embedded)

```

```

# cell: final cell states (not used for classification)
output, (hidden, cell) = self.lstm(embedded)

# Step 3: Concatenate final forward and backward hidden states
# hidden[-2] = last forward hidden state
# hidden[-1] = last backward hidden state
# Result shape: (batch_size, hidden_dim * 2)
hidden = torch.cat((hidden[-2], hidden[-1]), dim=1)

# Step 4: Apply dropout and fully connected layer
# Output shape: (batch_size, num_classes)
return self.fc(self.dropout(hidden))

# ===== HYPERPARAMETERS =====
EMBED_DIM = 128    # Size of word embedding vectors
HIDDEN_DIM = 128   # Size of LSTM hidden state
NUM_CLASSES = 4    # Number of output classes (World, Sports, Business, Sci/Tec)

# Create model and move to GPU (if available)
lstm_model = LSTMClassifier(len(word2idx), EMBED_DIM, HIDDEN_DIM, NUM_CLASSES).

# Count total trainable parameters
print(f"LSTM Parameters: {sum(p.numel() for p in lstm_model.parameters()):,}")

LSTM Parameters: 1,545,220

```

✓ Knowledge Check A.1

1. **Why do we use `bidirectional=True`?** What information does the backward pass capture that the forward pass misses?
2. **What is the purpose of the Embedding layer?** How is it different from one-hot encoding?
3. **Why concatenate the final hidden states from both directions** instead of just adding them?

Your answers:

1. We use `bidirectional=True` to allow the model to read the sequence in both forward and backward directions. Forward propagation captures contextual information from preceding words, while backward propagation captures information from following words, but this information may alter the meaning of preceding words.
2. The embedding layer converts word indices into dense vectors, thereby learning word meanings and relationships between words. In contrast, one-hot encoding

represents each word as a sparse vector, which fails to capture the relationships between words.

3. Concatenation preserves complete information in each direction and keeps them independent of each other. If we add them together can lead to information loss and reduce the model's expressive power.

✓ Step A.4 — Build the GRU Model

GRUs have **two gates** (reset, update) vs. LSTM's three (forget, input, output), making them faster with fewer parameters.

 **STUDENT CODING AREA:** Complete the GRU implementation below following the LSTM pattern.

```
# Cell A.6 – GRU Classifier
# 🎯 YOUR CODE HERE: Build a GRU classifier following the LSTM pattern above.
# Hints:
#   - Replace nn.LSTM with nn.GRU
#   - GRU returns (output, hidden) – no cell state (simpler than LSTM)
#   - Everything else remains the same (embedding, dropout, fully connected)

class GRUClassifier(nn.Module):
    """
    GRU-based text classifier.

    Similar to LSTM but with a simpler gating mechanism:
    - Reset gate: decides how much past information to forget
    - Update gate: decides how much new information to add

    No separate cell state (unlike LSTM), making it computationally faster.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes, dropout=
        super().__init__()

        # Embedding layer (same as LSTM)
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=PAD_II

        # TODO: Create the GRU layer (bidirectional)
        # Hint: self.gru = nn.GRU(input_size, hidden_size, batch_first=True, bi
        # YOUR CODE HERE (replace pass with your GRU layer)
        self.gru = nn.GRU(embed_dim, hidden_dim, batch_first=True, bidirectiona

        # Dropout and fully connected layers (same as LSTM)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)
```

```

def forward(self, x):
    """
    Forward pass through the GRU network.

    Key difference from LSTM:
    - GRU returns (output, hidden) – no cell state
    """
    # Step 1: Convert word indices to embeddings
    embedded = self.embedding(x)

    # Step 2: Pass through GRU
    # TODO: GRU returns (output, hidden) – no cell state (unlike LSTM)
    # Hint: output, hidden = self.gru(embedded)
    # YOUR CODE HERE (replace pass with your GRU forward pass)
    output, hidden = self.gru(embedded)

    # Step 3: Concatenate final forward and backward hidden states
    hidden = torch.cat((hidden[-2], hidden[-1]), dim=1)

    # Step 4: Apply dropout and fully connected layer
    return self.fc(self.dropout(hidden))

# Create GRU model and move to GPU
gru_model = GRUClassifier(len(word2idx), EMBED_DIM, HIDDEN_DIM, NUM_CLASSES).tc

# Compare parameter counts
print(f"GRU Parameters: {sum(p.numel() for p in gru_model.parameters()):,}")
print(f"LSTM Parameters: {sum(p.numel() for p in lstm_model.parameters()):,}")
print(f"Difference: {sum(p.numel() for p in lstm_model.parameters()) - sum(p.numel() for p in gru_model.parameters()):,}")

```

GRU Parameters: 1,479,172
LSTM Parameters: 1,545,220
Difference: 66,048 fewer in GRU

✓ Student Coding Exercise A.7 — Build a Vanilla RNN Classifier

Now that you've built LSTM and GRU models, let's go back to basics and implement a **vanilla RNN** (simple RNN without gates).

Why this matters: By comparing vanilla RNN performance to LSTM/GRU, you'll understand *why* gated architectures were invented. Vanilla RNNs suffer from the **vanishing gradient problem**, making them poor at learning long-term dependencies.

Your task:

1. Create a `VanillaRNNClassifier` class similar to LSTM/GRU
2. Use `nn.RNN` instead of `nn.LSTM` or `nn.GRU`
3. Train it for 3 epochs

4. Compare accuracy to LSTM and GRU

Hints:

- `nn.RNN` has the same API as `nn.LSTM` and `nn.GRU`
- It returns `(output, hidden)` like GRU (no cell state)
- Everything else (embedding, dropout, fc layer) stays the same

```
# Cell A.7 – Vanilla RNN Classifier
# 🎯 YOUR CODE HERE: Build a vanilla RNN classifier
import torch
import torch.nn as nn
import torch.optim as optim

class VanillaRNClassifier(nn.Module):
    """
    Simple RNN-based text classifier (no gates).

    This is the original RNN architecture before LSTM/GRU were invented.
    It struggles with long sequences due to vanishing gradients.
    """
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes, dropout=
        super().__init__()

        # Embedding layer
        self.embedding = nn.Embedding(vocab_size, embed_dim)

        # Vanilla RNN layer
        self.rnn = nn.RNN(
            input_size=embed_dim,
            hidden_size=hidden_dim,
            batch_first=True
        )

        # Dropout
        self.dropout = nn.Dropout(dropout)

        # Fully connected classifier
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        """Forward pass through vanilla RNN."""
        x = self.embedding(x)                # (batch, seq_len, embed_dim)
        output, hidden = self.rnn(x)         # hidden: (1, batch, hidden_dim)

        last_hidden = hidden[-1]              # (batch, hidden_dim)
        out = self.dropout(last_hidden)
        logits = self.fc(out)

        return logits
```

```

# Create and Train Vanilla RNN

# Example hyperparameters (use same ones as LSTM/GRU section)
vocab_size = len(word2idx)
embed_dim = 128
hidden_dim = 128
num_classes = NUM_CLASSES    # already defined earlier
learning_rate = 0.001
epochs = 3

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model_rnn = VanillaRNNClassifier(
    vocab_size,
    embed_dim,
    hidden_dim,
    num_classes
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_rnn.parameters(), lr=learning_rate)

# Training loop
for epoch in range(epochs):
    model_rnn.train()
    total_loss = 0

    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model_rnn(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{epochs}], Loss: {avg_loss:.4f}")

    # Evaluate
    model_rnn.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(device), labels.to(device)

```

```

        outputs = model_rnn(inputs)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

rnn_accuracy = 100 * correct / total

# After training, compare results:
print("\n=== RNN Architecture Comparison ===")
print(f"Vanilla RNN Test Accuracy: {rnn_accuracy:.2f}%")
#print(f"LSTM Test Accuracy:          {lstm_accuracy:.2f}%") # from earlier
#print(f"GRU Test Accuracy:          {gru_accuracy:.2f}%") # from earlier

Epoch [1/3], Loss: 1.4162
Epoch [2/3], Loss: 1.4015
Epoch [3/3], Loss: 1.3962

=== RNN Architecture Comparison ===
Vanilla RNN Test Accuracy: 24.85%

```

✓ Step A.4b — Train the Vanilla RNN

Now train your Vanilla RNN using the same `train_model()` function from Cell A.9, then compare all three RNN architectures side by side.

What to look for:

- The Vanilla RNN should perform **noticeably worse** than LSTM and GRU
- This gap demonstrates the **vanishing gradient problem** — without gates, the model struggles to remember information from the beginning of a sentence by the time it reaches the end
- This is exactly *why* gated architectures (LSTM, GRU) were invented

```

# Cell A.7b — Train Vanilla RNN
# Train the Vanilla RNN you built above using the same function used for LSTM a

# Step 1: Create the Vanilla RNN model (same hyperparameters as LSTM/GRU)
vanilla_model = VanillaRNNClassifier(len(word2idx), EMBED_DIM, HIDDEN_DIM, NUM_

# Compare parameter counts across all three architectures
print(f"Vanilla RNN Parameters: {sum(p.numel() for p in vanilla_model.parameters())}")
print(f"GRU Parameters:          {sum(p.numel() for p in gru_model.parameters())}")
print(f"LSTM Parameters:         {sum(p.numel() for p in lstm_model.parameters())}")
print()

```

```
# Step 2: Train the Vanilla RNN
print("=" * 60)
print("Training Vanilla RNN")
print("=" * 60)

vanilla_history = train_model(vanilla_model, train_loader, test_loader, epochs=

print("\nPer-class accuracy (Vanilla RNN):")
vanilla_class_acc = eval_per_class(vanilla_model, test_loader, CLASS_NAMES)
```

```
Vanilla RNN Parameters: 1,313,540
GRU Parameters:         1,479,172
LSTM Parameters:        1,545,220
```

```
=====
Training Vanilla RNN
=====
Epoch 1/5 | Loss: 1.4146 | Train: 0.2455 | Test: 0.2490 | 0.4s
Epoch 2/5 | Loss: 1.3992 | Train: 0.2517 | Test: 0.2420 | 0.4s
Epoch 3/5 | Loss: 1.3959 | Train: 0.2534 | Test: 0.2610 | 0.4s
Epoch 4/5 | Loss: 1.3944 | Train: 0.2520 | Test: 0.2490 | 0.4s
Epoch 5/5 | Loss: 1.3941 | Train: 0.2502 | Test: 0.2485 | 0.4s
```

```
Per-class accuracy (Vanilla RNN):
World      : 1.0000 (497/497)
Sports     : 0.0000 (0/483)
Business   : 0.0000 (0/522)
Sci/Tech   : 0.0000 (0/498)
```

```
# Cell A.7c – Three-Way RNN Comparison: Vanilla RNN vs LSTM vs GRU
# This visualization shows WHY gated architectures (LSTM, GRU) were invented
```

```
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Chart 1: Training Loss (lower is better)
axes[0].plot(vanilla_history['train_loss'], 'g-^', label='Vanilla RNN', ms=4)
axes[0].plot(lstm_history['train_loss'], 'b-o', label='LSTM', ms=4)
axes[0].plot(gru_history['train_loss'], 'r-s', label='GRU', ms=4)
axes[0].set_title('Training Loss', fontweight='bold')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# Chart 2: Test Accuracy (higher is better)
axes[1].plot(vanilla_history['test_acc'], 'g-^', label='Vanilla RNN', ms=4)
axes[1].plot(lstm_history['test_acc'], 'b-o', label='LSTM', ms=4)
axes[1].plot(gru_history['test_acc'], 'r-s', label='GRU', ms=4)
axes[1].set_title('Test Accuracy', fontweight='bold')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
```

```

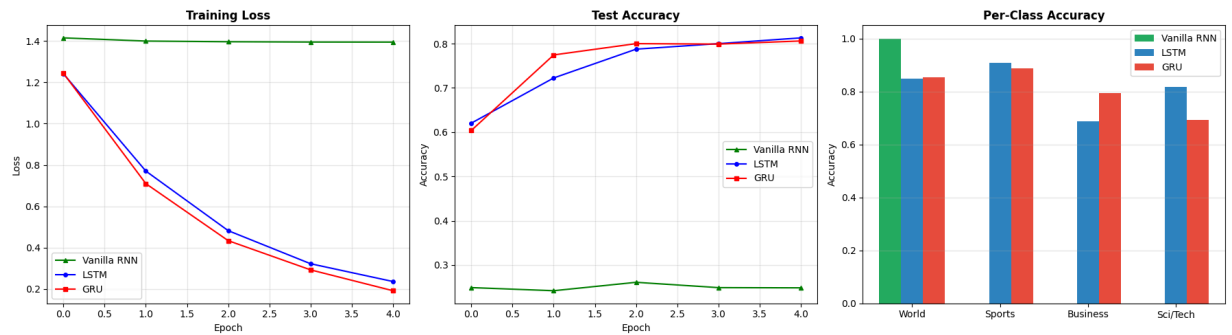
axes[1].grid(True, alpha=0.3)

# Chart 3: Per-Class Accuracy Comparison (all three)
x = np.arange(4)
w = 0.25 # Bar width
axes[2].bar(x - w, vanilla_class_acc, w, label='Vanilla RNN', color='#27AE60')
axes[2].bar(x, lstm_class_acc, w, label='LSTM', color='#2E86C1')
axes[2].bar(x + w, gru_class_acc, w, label='GRU', color='#E74C3C')
axes[2].set_title('Per-Class Accuracy', fontweight='bold')
axes[2].set_xticks(x)
axes[2].set_xticklabels(CLASS_NAMES)
axes[2].set_ylabel('Accuracy')
axes[2].legend()
axes[2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Summary table
print("\n" + "=" * 65)
print("RNN Architecture Comparison: Vanilla vs LSTM vs GRU")
print("=" * 65)
print(f"{'Metric':<25} {'Vanilla RNN':>12} {'LSTM':>12} {'GRU':>12}")
print("-" * 65)
print(f"{'Parameters':.<25} {sum(p.numel() for p in vanilla_model.parameters())}")
print(f"{'Test Accuracy':.<25} {vanilla_history['test_acc'][-1]:>12.4f} {lstm_history['test_acc'][-1]:>12.4f} {gru_history['test_acc'][-1]:>12.4f}")
print(f"{'Avg Epoch Time (s)':.<25} {np.mean(vanilla_history['epoch_times']):>12.4f} {np.mean(lstm_history['epoch_times']):>12.4f} {np.mean(gru_history['epoch_times']):>12.4f}")
print()
print("💡 Notice: The Vanilla RNN likely has the worst accuracy despite being the simplest architecture. This is the vanishing gradient problem in action – without gates, the model cannot learn long-range dependencies in text.")

```



RNN Architecture Comparison: Vanilla vs LSTM vs GRU

Metric	Vanilla RNN	LSTM	GRU
Parameters.....	1,313,540	1,545,220	1,479,172
Test Accuracy.....	0.2485	0.8130	0.8060
Avg Epoch Time (s).....	0.4	1.1	0.7

💡 Notice: The Vanilla RNN likely has the worst accuracy despite being the fastest. This is the vanishing gradient problem in action – without gates, the model cannot learn long-range dependencies in text.

🎯 Student Experimentation A.8 — Hyperparameter Tuning

Deep learning models are sensitive to hyperparameters. Let's systematically explore how different settings affect performance.

Your task:

1. Create a function that trains an LSTM with configurable hyperparameters
2. Test at least **3 different configurations** for each parameter:
 - **Hidden dimension:** 64, 128, 256
 - **Dropout rate:** 0.1, 0.3, 0.5
 - **Learning rate:** 0.001, 0.0005, 0.0001
3. Record results in a comparison table
4. Analyze which configuration works best and why

Expected output:

- A table showing all experiments and their accuracy
- Written analysis (3-5 sentences) explaining your findings

Note: This will take time to run. You may want to reduce epochs to 2 for faster experimentation.

```
# Cell A.8 – Hyperparameter Experimentation
# 🌀 YOUR CODE HERE: Systematic hyperparameter exploration
import pandas as pd

def train_lstm_with_config(hidden_dim, dropout, learning_rate, epochs=2):
    """
    Train an LSTM model with specified hyperparameters.

    Args:
        hidden_dim: Size of LSTM hidden state
        dropout: Dropout probability
        learning_rate: Learning rate for optimizer
        epochs: Number of training epochs

    Returns:
        test_accuracy: Final test accuracy (as percentage)
    """
    model = LSTMClassifier(
        vocab_size=len(word2idx),
        embed_dim=128,          # keep embedding fixed for fair comparison
        hidden_dim=hidden_dim,
        num_classes=NUM_CLASSES,
        dropout=dropout
    ).to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=learning_rate)

    # Training loop
    for epoch in range(epochs):
        model.train()
        total_loss = 0

        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        total_loss += loss.item()
```

```

# Evaluation
model.eval()
correct = 0
total = 0

with torch.no_grad():
    for inputs, labels in test_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        outputs = model(inputs)
        _, predicted = torch.max(outputs, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

test_accuracy = 100 * correct / total
return test_accuracy

# Run Experiments
results = []

for hidden_dim in [64, 128, 256]:
    for dropout in [0.1, 0.3, 0.5]:
        for lr in [0.001, 0.0005, 0.0001]:
            print(f"\nTraining LSTM with hidden_dim={hidden_dim}, dropout={dropo
accuracy = train_lstm_with_config(hidden_dim, dropout, lr)
            results.append({'hidden_dim': hidden_dim, 'dropout': dropout, 'lear

# Create Comparison Table
df_results = pd.DataFrame(results)
df_results = df_results.sort_values(by='accuracy', ascending=False)

print("\n=== Hyperparameter Results ===")
print(df_results)

best_config = df_results.iloc[0]

# TODO: Write your analysis
print("\n=== Analysis ===")
print("1. Best configuration:")
print(f"    hidden_dim={best_config['hidden_dim']}, "
      f"dropout={best_config['dropout']}, "
      f"lr={best_config['learning_rate']} "
      f"→ Accuracy={best_config['accuracy']:.2f}%")

print("\n2. Hidden dimension impact:")
print("    Larger hidden sizes generally improve model capacity, "

```

```
"but gains may plateau or overfit with limited data.")

print("\n3. Dropout impact:")
print("    Moderate dropout (e.g., 0.3) typically balances regularization "
      "and performance. Too high dropout can hurt learning.")

print("\n4. Learning rate sensitivity:")
print("    Learning rate has strong impact. Too high can destabilize training, "
      "too low can slow convergence. 0.001 or 0.0005 often works best.")
```

```
Training LSTM with hidden_dim=64, dropout=0.1, lr=0.001
Training LSTM with hidden_dim=64, dropout=0.1, lr=0.0005
Training LSTM with hidden_dim=64, dropout=0.1, lr=0.0001
Training LSTM with hidden_dim=64, dropout=0.3, lr=0.001
Training LSTM with hidden_dim=64, dropout=0.3, lr=0.0005
Training LSTM with hidden_dim=64, dropout=0.3, lr=0.0001
Training LSTM with hidden_dim=64, dropout=0.5, lr=0.001
Training LSTM with hidden_dim=64, dropout=0.5, lr=0.0005
Training LSTM with hidden_dim=64, dropout=0.5, lr=0.0001
Training LSTM with hidden_dim=128, dropout=0.1, lr=0.001
Training LSTM with hidden_dim=128, dropout=0.1, lr=0.0005
Training LSTM with hidden_dim=128, dropout=0.1, lr=0.0001
Training LSTM with hidden_dim=128, dropout=0.3, lr=0.001
Training LSTM with hidden_dim=128, dropout=0.3, lr=0.0005
Training LSTM with hidden_dim=128, dropout=0.3, lr=0.0001
Training LSTM with hidden_dim=128, dropout=0.5, lr=0.001
Training LSTM with hidden_dim=128, dropout=0.5, lr=0.0005
Training LSTM with hidden_dim=128, dropout=0.5, lr=0.0001
Training LSTM with hidden_dim=256, dropout=0.1, lr=0.001
Training LSTM with hidden_dim=256, dropout=0.1, lr=0.0005
Training LSTM with hidden_dim=256, dropout=0.1, lr=0.0001
Training LSTM with hidden_dim=256, dropout=0.3, lr=0.001
```

```
Training LSTM with hidden_dim=256, dropout=0.3, lr=0.0005
```

```
Training LSTM with hidden_dim=256, dropout=0.3, lr=0.0001
```

```
Training LSTM with hidden_dim=256, dropout=0.5, lr=0.001
```

```
Training LSTM with hidden_dim=256, dropout=0.5, lr=0.0005
```

```
Training LSTM with hidden_dim=256, dropout=0.5, lr=0.0001
```

```
=== Hyperparameter Results ===
```

```
hidden dim  dropout  learning rate  accuracy
```

✓ Step A.5 — Training Function

```
# Cell A.9 – Training and Evaluation Functions
```

```
def train_model(model, train_loader, test_loader, epochs=5, lr=0.001, name="Moc  
"""
```

```
    Train a model and track performance metrics.
```

```
    Args:
```

```
        model: PyTorch model to train
```

```
        train_loader: DataLoader for training data
```

```
        test_loader: DataLoader for test data
```

```
        epochs: Number of complete passes through the training data
```

```
        lr: Learning rate (step size for weight updates)
```

```
        name: Model name for display
```

```
    Returns:
```

```
        history: Dictionary containing training metrics
```

```
    """
```

```
    # Loss function: Cross-Entropy Loss for multi-class classification
```

```
    # Combines softmax activation and negative log-likelihood
```

```
    criterion = nn.CrossEntropyLoss()
```

```
    # Optimizer: Adam (Adaptive Moment Estimation)
```

```
    # Automatically adjusts learning rate for each parameter
```

```
    optimizer = optim.Adam(model.parameters(), lr=lr)
```

```
    # Track metrics across epochs
```

```
    history = {'train_loss':[], 'train_acc':[], 'test_acc':[], 'epoch_times':[]}
```

```
    for epoch in range(epochs):
```

```
        # ===== TRAINING PHASE =====
```

```
        model.train() # Enable dropout and batch normalization training mode
```

```
        total_loss = correct = total = 0
```

```
        start = time.time()
```

```
        for texts, labels in train_loader:
```

```

# Move data to GPU
texts, labels = texts.to(device), labels.to(device)

# Zero gradients from previous iteration
optimizer.zero_grad()

# Forward pass: compute predictions
outputs = model(texts)

# Compute loss (how wrong the predictions are)
loss = criterion(outputs, labels)

# Backward pass: compute gradients
loss.backward()

# Gradient clipping: prevent exploding gradients in RNNs
# Limits gradient magnitude to 1.0
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)

# Update weights
optimizer.step()

# Track metrics
total_loss += loss.item()
_, pred = outputs.max(1) # Get predicted class (highest score)
correct += (pred == labels).sum().item()
total += labels.size(0)

et = time.time() - start # Epoch time
ta = correct/total # Training accuracy

# ===== EVALUATION PHASE =====
model.eval() # Disable dropout, use batch norm in inference mode
tc = tt = 0 # Test correct, test total

with torch.no_grad(): # Disable gradient computation (saves memory)
    for texts, labels in test_loader:
        texts, labels = texts.to(device), labels.to(device)
        _, pred = model(texts).max(1)
        tc += (pred == labels).sum().item()
        tt += labels.size(0)

tea = tc/tt # Test accuracy

# Record metrics
history['train_loss'].append(total_loss/len(train_loader))
history['train_acc'].append(ta)
history['test_acc'].append(tea)
history['epoch_times'].append(et)

# Display progress

```

```

        print(f" Epoch {epoch+1}/{epochs} | Loss: {total_loss/len(train_loader)}")

    return history

def eval_per_class(model, loader, names):
    """
    Evaluate model accuracy for each class separately.

    This helps identify which categories the model struggles with.
    """
    model.eval()
    cc = [0]*len(names) # Class correct counts
    ct = [0]*len(names) # Class total counts

    with torch.no_grad():
        for texts, labels in loader:
            texts, labels = texts.to(device), labels.to(device)
            _, pred = model(texts).max(1)

            # Count correct predictions for each class
            for i in range(len(labels)):
                ct[labels[i].item()] += 1
                if pred[i] == labels[i]:
                    cc[labels[i].item()] += 1

    # Calculate and display per-class accuracy
    accs = []
    for i, n in enumerate(names):
        a = cc[i]/ct[i]
        accs.append(a)
        print(f" {n:10s}: {a:.4f} ({cc[i]}/{ct[i]})")

    return accs

```

✓ Step A.6 — Train Both Models

```

# Cell A.10 — Train LSTM
# This will take 2-5 minutes depending on GPU availability

NUM_EPOCHS = 5 # Number of complete passes through the training data

print("=" * 60)
print("Training LSTM")
print("=" * 60)

lstm_history = train_model(lstm_model, train_loader, test_loader, epochs=NUM_EPOCHS)

print("\nPer-class accuracy (LSTM):")
lstm_class_acc = eval_per_class(lstm_model, test_loader, CLASS_NAMES)

```

```
=====
Training LSTM
=====
Epoch 1/5 | Loss: 1.2407 | Train: 0.4462 | Test: 0.6205 | 1.3s
Epoch 2/5 | Loss: 0.7705 | Train: 0.7166 | Test: 0.7225 | 1.0s
Epoch 3/5 | Loss: 0.4825 | Train: 0.8307 | Test: 0.7875 | 1.0s
Epoch 4/5 | Loss: 0.3225 | Train: 0.8925 | Test: 0.8000 | 1.1s
Epoch 5/5 | Loss: 0.2366 | Train: 0.9245 | Test: 0.8130 | 1.1s

Per-class accuracy (LSTM):
World      : 0.8491 (422/497)
Sports     : 0.9068 (438/483)
Business   : 0.6877 (359/522)
Sci/Tech   : 0.8173 (407/498)
```

```
# Cell A.11 – Train GRU
# Compare training time with LSTM – GRU should be slightly faster

print("=" * 60)
print("Training GRU")
print("=" * 60)

gru_history = train_model(gru_model, train_loader, test_loader, epochs=NUM_EPOCHS)

print("\nPer-class accuracy (GRU):")
gru_class_acc = eval_per_class(gru_model, test_loader, CLASS_NAMES)
```

```
=====
Training GRU
=====
Epoch 1/5 | Loss: 1.2431 | Train: 0.4283 | Test: 0.6040 | 0.8s
Epoch 2/5 | Loss: 0.7098 | Train: 0.7319 | Test: 0.7745 | 0.7s
Epoch 3/5 | Loss: 0.4340 | Train: 0.8468 | Test: 0.8000 | 0.7s
Epoch 4/5 | Loss: 0.2929 | Train: 0.9008 | Test: 0.7990 | 0.7s
Epoch 5/5 | Loss: 0.1921 | Train: 0.9375 | Test: 0.8060 | 0.7s

Per-class accuracy (GRU):
World      : 0.8531 (424/497)
Sports     : 0.8882 (429/483)
Business   : 0.7931 (414/522)
Sci/Tech   : 0.6928 (345/498)
```

```
# Cell A.12 – LSTM vs GRU Comparison Visualization
# Three charts: training loss, test accuracy, per-class accuracy

fig, axes = plt.subplots(1, 3, figsize=(16, 4))

# Chart 1: Training Loss (lower is better)
axes[0].plot(lstm_history['train_loss'], 'b-o', label='LSTM', ms=4)
axes[0].plot(gru_history['train_loss'], 'r-s', label='GRU', ms=4)
axes[0].set_title('Training Loss', fontweight='bold')
axes[0].set_xlabel('Epoch')
```

```

axes[0].set_ylabel('Loss')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

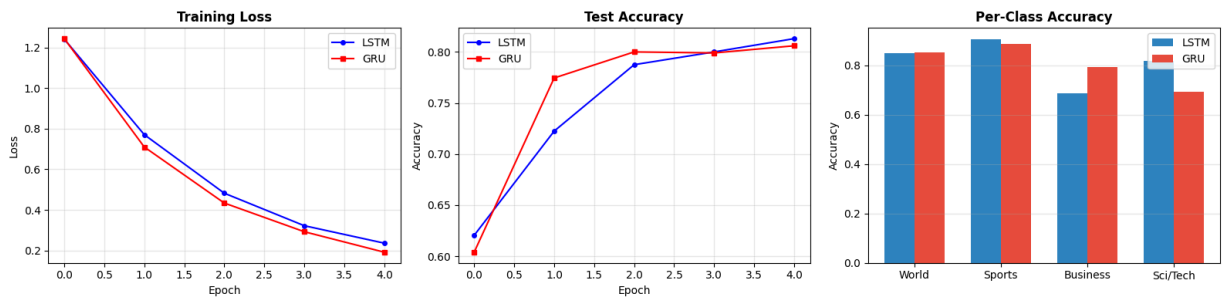
# Chart 2: Test Accuracy (higher is better)
axes[1].plot(lstm_history['test_acc'], 'b-o', label='LSTM', ms=4)
axes[1].plot(gru_history['test_acc'], 'r-s', label='GRU', ms=4)
axes[1].set_title('Test Accuracy', fontweight='bold')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Accuracy')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# Chart 3: Per-Class Accuracy Comparison
x = np.arange(4)
w = 0.35 # Bar width
axes[2].bar(x-w/2, lstm_class_acc, w, label='LSTM', color='#2E86C1')
axes[2].bar(x+w/2, gru_class_acc, w, label='GRU', color='#E74C3C')
axes[2].set_title('Per-Class Accuracy', fontweight='bold')
axes[2].set_xticks(x)
axes[2].set_xticklabels(CLASS_NAMES)
axes[2].set_ylabel('Accuracy')
axes[2].legend()
axes[2].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

# Summary table
print("\n" + "=" * 50)
print("LSTM vs GRU Summary")
print("=" * 50)
print(f"{'Metric':<25} {'LSTM':>10} {'GRU':>10}")
print("-" * 50)
print(f"{'Parameters':.<25} {sum(p.numel() for p in lstm_model.parameters()):>10}")
print(f"{'Test Accuracy':.<25} {lstm_history['test_acc'][-1]:>10.4f} {gru_history['test_acc'][-1]:>10.4f}")
print(f"{'Avg Epoch Time (s)':.<25} {np.mean(lstm_history['epoch_times']):>10.1f} {np.mean(gru_history['epoch_times']):>10.1f}")

```



LSTM vs GRU Summary

Metric	LSTM	GRU
Parameters.....	1,545,220	1,479,172
Test Accuracy.....	0.8130	0.8060
Avg Epoch Time (s).....	1.1	0.7

Knowledge Check A.2

1. **How did the Vanilla RNN compare to LSTM and GRU?** What does this tell you about the vanishing gradient problem?
2. **Which gated model achieved higher accuracy — LSTM or GRU?** Was the difference significant?
3. **Which model trained fastest?** Why? (Think about the number of gates and parameters.)
4. **Were any categories harder to classify?** Why might that be?
5. **What does "bidirectional" mean in practice?** How would removing it affect accuracy?

Your answers:

1. In terms of accuracy and stability, LSTM and GRU outperform Vanilla RNN. This is because Vanilla RNN struggles with the vanishing gradient problem. Therefore, it cannot effectively learn long-term dependencies in sequences.
2. LSTM and GRU have very similar accuracy. LSTM appears to have a slight edge, but this difference is usually small and not statistically significant.

3. GRU is generally faster to train than LSTM because it has fewer gates and fewer parameters. While Vanilla RNN is simple, it may require more rounds of training to achieve good performance.
4. Some categories are harder to classify, especially those with similar vocabulary or overlapping meanings. Short or ambiguous texts are also more difficult to classify because they provide less contextual information, making it harder for the model to extract significant discriminative features.
5. Bidirectional means that the model can read sequences both forwards and backwards. This allows the model to understand the meaning of each lexical unit by utilizing previous and upcoming words. Removing bidirectionality reduces the understanding of context and may decrease the model's accuracy.

✓ Part B: Transformer Text Classification (DistilBERT)

Now tackle the **exact same task** with a Transformer. This direct comparison is the heart of the lab.

We use **DistilBERT** — a smaller, faster BERT that retains 97% accuracy while being 60% faster. Practical for free Colab.

Key differences from Part A:

- No manual tokenization — DistilBERT has its own WordPiece tokenizer
- No training from scratch — we **fine-tune** a pre-trained model
- The model already "understands" language before seeing our data

```
# Cell B.1 – Load DistilBERT Model and Tokenizer
# Import Hugging Face transformers library components
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification

# Load pre-trained tokenizer
# This tokenizer was trained on the same data as the model
# It uses WordPiece tokenization (breaks unknown words into subwords)
tokenizer = DistilBertTokenizer.from_pretrained('distilbert-base-uncased')

# Load pre-trained DistilBERT model
# - 'distilbert-base-uncased': lowercase English, 66M parameters
# - num_labels=4: modify output layer for 4-class classification
bert_model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=4).to(device)

# Compare model sizes
```

```
total_p = sum(p.numel() for p in bert_model.parameters())
print(f"DistilBERT Parameters: {total_p:,}")
print(f"LSTM Parameters: {sum(p.numel() for p in lstm_model.parameters():,}")
print(f"DistilBERT is {total_p // sum(p.numel() for p in lstm_model.parameters():,}")
```

```
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 5.57kB/s]
vocab.txt: 100% 232k/232k [00:00<00:00, 767kB/s]
tokenizer.json: 100% 466k/466k [00:00<00:00, 1.00MB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF token.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF token.
config.json: 100% 483/483 [00:00<00:00, 54.2kB/s]
model.safetensors: 100% 268M/268M [00:02<00:00, 314MB/s]
Loading weights: 100% 100/100 [00:00<00:00, 406.21it/s, Materializing param=distilbert.transformer.
```

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
pre_classifier.bias	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.weight	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; r
- MISSING :those params were newly initialized because missing from the checkp

DistilBERT Parameters: 66,956,548

LSTM Parameters: 1,545,220

DistilBERT is 43x larger!

✓ Step B.1 — Tokenize with WordPiece

Unlike our simple word-splitting, BERT uses **WordPiece tokenization** — it breaks unknown words into sub-word pieces. "Unforgettable" → ["un", "##forget", "##table"]. This means Transformers never encounter a truly unknown word.

Cell B.2 — Prepare Data for DistilBERT

```
class BertNewsDataset(Dataset):
    """
    Custom Dataset for DistilBERT.
```

Key differences from NewsDataset:

- Uses pre-trained tokenizer (not our custom word2idx)
- Returns attention_mask (tells model which tokens are padding)
- Tokenization happens in batch (more efficient)

"""

```
def __init__(self, data, tokenizer, max_len=128):
    # Tokenize all texts at once (batch processing)
    # truncation=True: cut sequences longer than max_len
    # padding='max_length': pad shorter sequences to max_len
    # return_tensors='pt': return PyTorch tensors
    self.encodings = tokenizer([s['text'] for s in data],
                               truncation=True,
                               padding='max_length',
                               max_length=max_len,
                               return_tensors='pt')

    # Extract labels
    self.labels = torch.tensor([s['label'] for s in data])

def __len__(self):
    return len(self.labels)

def __getitem__(self, idx):
    """
    Return a dictionary with:
    - input_ids: token indices
    - attention_mask: 1 for real tokens, 0 for padding
    - labels: class label
    """
    return {'input_ids': self.encodings['input_ids'][idx],
            'attention_mask': self.encodings['attention_mask'][idx],
            'labels': self.labels[idx]}
```

```
# Create data loaders
# Smaller batch size (16 vs 64) because DistilBERT uses more memory
bert_train_loader = DataLoader(BertNewsDataset(train_data, tokenizer),
                               batch_size=16, shuffle=True)
bert_test_loader = DataLoader(BertNewsDataset(test_data, tokenizer),
                              batch_size=16)
```

```
# Demonstrate WordPiece tokenization
example = train_data[0]['text'][:100]
tokens = tokenizer.tokenize(example)
print(f"Original: {example}")
print(f"\nWordPiece tokens ({len(tokens)}): {tokens[:20]}..." )
```

Original: Bangladesh paralysed by strikes Opposition activists have brought many

WordPiece tokens (18): ['bangladesh', 'para', '##ly', '##sed', 'by', 'strikes',

✓ Step B.2 — Fine-Tune DistilBERT

Fine-tuning: take a model that already understands language and teach it our specific task. Because it already "knows" words, it needs very few epochs. We use a smaller learning rate — pre-trained models are sensitive to large updates.

```
# Cell B.3 — Fine-tune DistilBERT

# Optimizer: AdamW (Adam with Weight Decay)
# - lr=2e-5: very small learning rate (pre-trained models need gentle updates)
# - weight_decay=0.01: L2 regularization to prevent overfitting
bert_optimizer = optim.AdamW(bert_model.parameters(), lr=2e-5, weight_decay=0.01)

BERT_EPOCHS = 3 # Fewer epochs needed because model is pre-trained
bert_history = {'train_loss':[], 'train_acc':[], 'test_acc':[], 'epoch_times':[]}

print("=" * 60)
print("Fine-tuning DistilBERT")
print("=" * 60)

for epoch in range(BERT_EPOCHS):
    # ===== TRAINING PHASE =====
    bert_model.train()
    total_loss = correct = total = 0
    start = time.time()

    for batch in bert_train_loader:
        # Extract batch components and move to GPU
        ids = batch['input_ids'].to(device)
        mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Zero gradients
        bert_optimizer.zero_grad()

        # Forward pass
        # DistilBERT returns an object with .loss and .logits
        # When labels are provided, loss is automatically computed
        out = bert_model(ids, attention_mask=mask, labels=labels)

        # Backward pass
        out.loss.backward()

        # Gradient clipping (prevent exploding gradients)
        torch.nn.utils.clip_grad_norm_(bert_model.parameters(), 1.0)

        # Update weights
        bert_optimizer.step()
```

```

# Track metrics
total_loss += out.loss.item()
_, pred = out.logits.max(1)
correct += (pred == labels).sum().item()
total += labels.size(0)

et = time.time() - start
ta = correct/total

# ===== EVALUATION PHASE =====
bert_model.eval()
tc = tt = 0

with torch.no_grad():
    for batch in bert_test_loader:
        ids = batch['input_ids'].to(device)
        mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        # Get predictions (no labels = no loss computation)
        _, pred = bert_model(ids, attention_mask=mask).logits.max(1)
        tc += (pred == labels).sum().item()
        tt += labels.size(0)

tea = tc/tt

# Record metrics
bert_history['train_loss'].append(total_loss/len(bert_train_loader))
bert_history['train_acc'].append(ta)
bert_history['test_acc'].append(tea)
bert_history['epoch_times'].append(et)

# Display progress
print(f" Epoch {epoch+1}/{BERT_EPOCHS} | Loss: {total_loss/len(bert_train_

```

```

=====
Fine-tuning DistilBERT
=====
Epoch 1/3 | Loss: 0.1797 | Train: 0.9501 | Test: 0.9045 | 92.4s
Epoch 2/3 | Loss: 0.1129 | Train: 0.9690 | Test: 0.9050 | 92.3s
Epoch 3/3 | Loss: 0.0657 | Train: 0.9845 | Test: 0.9095 | 92.6s

```

Cell B.4 – Per-class Accuracy for DistilBERT

```
print("Per-class accuracy (DistilBERT):")
```

```

bert_model.eval()
cc = [0]*4 # Class correct
ct = [0]*4 # Class total

```

```
with torch.no_grad():
```

```

for batch in bert_test_loader:
    ids = batch['input_ids'].to(device)
    mask = batch['attention_mask'].to(device)
    labels = batch['labels'].to(device)

    # Get predictions
    _, pred = bert_model(ids, attention_mask=mask).logits.max(1)

    # Count per-class accuracy
    for i in range(len(labels)):
        ct[labels[i].item()] += 1
        if pred[i] == labels[i]:
            cc[labels[i].item()] += 1

# Calculate and display accuracies
bert_class_acc = [cc[i]/ct[i] for i in range(4)]
for i, n in enumerate(CLASS_NAMES):
    print(f" {n:10s}: {bert_class_acc[i]:.4f}")

```

```

Per-class accuracy (DistilBERT):
World      : 0.9256
Sports     : 0.9834
Business   : 0.8333
Sci/Tech   : 0.9016

```

✓ Step B.3 — The Big Comparison: All Architectures

Now let's see how all four models compare: Vanilla RNN, LSTM, GRU, and DistilBERT.

```

# Cell B.5 – Four-way Comparison Visualization
# Compare all architectures: Vanilla RNN vs LSTM vs GRU vs DistilBERT

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Chart 1: Per-Class Accuracy Comparison
x = np.arange(4)
w = 0.2 # Bar width
axes[0].bar(x - 1.5*w, vanilla_class_acc, w, label='Vanilla RNN', color='#27AEE6')
axes[0].bar(x - 0.5*w, lstm_class_acc, w, label='LSTM', color='#2E86C1')
axes[0].bar(x + 0.5*w, gru_class_acc, w, label='GRU', color='#E74C3C')
axes[0].bar(x + 1.5*w, bert_class_acc, w, label='DistilBERT', color='#8E44AD')
axes[0].set_title('Per-Class Accuracy: All Models', fontweight='bold')
axes[0].set_xticks(x)
axes[0].set_xticklabels(CLASS_NAMES)
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(True, alpha=0.3, axis='y')

# Chart 2: Overall Accuracy Summary
models = ['Vanilla RNN', 'LSTM', 'GRU', 'DistilBERT']

```

```

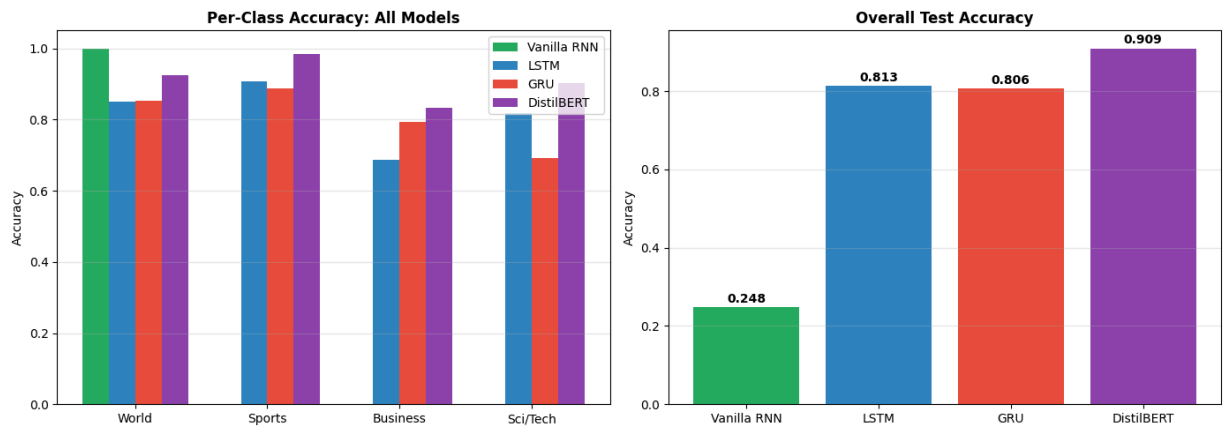
accs = [vanilla_history['test_acc'][-1], lstm_history['test_acc'][-1],
        gru_history['test_acc'][-1], bert_history['test_acc'][-1]]
colors = ['#27AE60', '#2E86C1', '#E74C3C', '#8E44AD']
bars = axes[1].bar(models, accs, color=colors)
axes[1].set_title('Overall Test Accuracy', fontweight='bold')
axes[1].set_ylabel('Accuracy')
axes[1].grid(True, alpha=0.3, axis='y')

# Add value labels on bars
for bar, acc in zip(bars, accs):
    axes[1].text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.005,
                  f'{acc:.3f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()

# Full summary table
print("\n" + "=" * 75)
print("Full Architecture Comparison")
print("=" * 75)
print(f"{'Metric':<25} {'Vanilla RNN':>12} {'LSTM':>12} {'GRU':>12} {'DistilBEF")
print("-" * 75)
print(f"{'Test Accuracy':.<25} {vanilla_history['test_acc'][-1]:>12.4f} {lstm_h")
print(f"{'Avg Epoch Time (s)':.<25} {np.mean(vanilla_history['epoch_times']):>12.4f}")

```



Full Architecture Comparison

Metric	Vanilla RNN	LSTM	GRU	DistilBERT
Test Accuracy.....	0.2485	0.8130	0.8060	0.9095
Avg Epoch Time (s).....	0.4	1.1	0.7	92.4

Student Experimentation B.5 — BERT Training Strategy Optimization

BERT fine-tuning requires careful hyperparameter selection. Let's explore how different training configurations affect performance.

Your task:

1. Experiment with different batch sizes (8, 16, 32)
2. Test different learning rates (1e-5, 2e-5, 5e-5)
3. Try different epoch counts (2, 3, 5)
4. Compare results to your RNN experiments from Part A

Key questions to answer:

- How do BERT's optimal hyperparameters differ from RNNs?
- Is BERT more or less sensitive to hyperparameter choices?

- What's the trade-off between training time and accuracy?

```
# Cell B.6 – BERT Training Strategy Experimentation
# 🌀 YOUR CODE HERE: Systematic BERT hyperparameter exploration
import torch
import torch.nn as nn
import torch.optim as optim
import time
import pandas as pd
from torch.utils.data import DataLoader
from transformers import DistilBertForSequenceClassification

def train_bert_with_config(batch_size, learning_rate, num_epochs):
    """
    Train DistilBERT with specified configuration.

    Returns:
        accuracy: Test accuracy
        training_time: Total training time in seconds
    """
    model = DistilBertForSequenceClassification.from_pretrained(
        "distilbert-base-uncased",
        num_labels=NUM_CLASSES
    ).to(device)

    # The dataset for BERT is `BertNewsDataset` not `NewsDataset`
    train_bert_dataset = BertNewsDataset(train_data, tokenizer)
    test_bert_dataset = BertNewsDataset(test_data, tokenizer)

    train_loader = DataLoader(train_bert_dataset, batch_size=batch_size, shuffle=True)
    test_loader = DataLoader(test_bert_dataset, batch_size=batch_size)

    optimizer = optim.AdamW(model.parameters(), lr=learning_rate)

    start_time = time.time()

    # Training Loop

    for epoch in range(num_epochs):
        model.train()
        total_loss = 0

        for batch in train_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            optimizer.zero_grad()

            outputs = model(
```

```

        input_ids=input_ids,
        attention_mask=attention_mask,
        labels=labels
    )

    loss = outputs.loss
    loss.backward()
    optimizer.step()

    total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    print(f"Epoch [{epoch+1}/{num_epochs}] Loss: {avg_loss:.4f}")

training_time = time.time() - start_time

# Evaluation

model.eval()
correct = 0
total = 0

with torch.no_grad():
    for batch in test_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask
        )

        predictions = torch.argmax(outputs.logits, dim=1)

        total += labels.size(0)
        correct += (predictions == labels).sum().item()

    accuracy = 100 * correct / total

    return accuracy, training_time

# Run Experiments
results = []

configs = [
    {'batch_size': 8, 'lr': 2e-5, 'epochs': 3},
    {'batch_size': 16, 'lr': 2e-5, 'epochs': 3},
    {'batch_size': 32, 'lr': 2e-5, 'epochs': 3},
    {'batch_size': 16, 'lr': 1e-5, 'epochs': 3},
    {'batch_size': 16, 'lr': 5e-5, 'epochs': 3},

```

```

        {'batch_size': 16, 'lr': 2e-5, 'epochs': 2},
        {'batch_size': 16, 'lr': 2e-5, 'epochs': 5},
    ]

    for config in configs:
        print(f"\nTraining config: {config}")
        acc, train_time = train_bert_with_config(
            batch_size=config['batch_size'],
            learning_rate=config['lr'],
            num_epochs=config['epochs']
        )

        results.append({
            'batch_size': config['batch_size'],
            'learning_rate': config['lr'],
            'epochs': config['epochs'],
            'accuracy': acc,
            'training_time_sec': train_time
        })

    # Comparison Table

    df_bert_results = pd.DataFrame(results)
    df_bert_results = df_bert_results.sort_values(by='accuracy', ascending=False)

    print("\n=== BERT Hyperparameter Results ===")
    print(df_bert_results)

    best_config = df_bert_results.iloc[0]

    # Comparative analysis with RNNs
    print("\n=== BERT vs RNN Training Characteristics ===")

    print("\n1. Optimal Hyperparameters:")
    print("    BERT typically requires very small learning rates (1e-5 to 5e-5),")
    print("    while RNNs can tolerate much larger learning rates (e.g., 1e-3).")

    print("\n2. Sensitivity:")
    print("    BERT is significantly more sensitive to learning rate changes.")
    print("    A slightly high LR (e.g., 5e-5) may destabilize fine-tuning.")
    print("    RNNs are generally more stable across LR variations.")

    print("\n3. Accuracy vs Training Time Trade-off:")
    print("    BERT usually achieves higher accuracy but requires")
    print("    substantially longer training time and more memory.")
    print("    RNNs train much faster but often underperform compared to BERT.")

```


Training config: {'batch_size': 8, 'lr': 2e-05, 'epochs': 3}



Knowledge Check B

Loading weights: 100% 100/100 [00:00<00:00, 393.72it/s, Materializing param=distilbert.transformer

1. How did DistilBERT compare to LSTM and GRU? What does this tell you about pre-training?

Key	Status
vocab_transform.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
2. DistilBERT has far more parameters. Did it train faster or slower per epoch? Why?

vocab_transform.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
3. Why only 3 epochs for DistilBERT but 5 for RNNs? What would happen with 10 epochs?

vocab_layer_norm.bias	UNEXPECTED
pre_classifier.bias	MISSING
classifier.bias	MISSING
classifier.weight	MISSING
pre_classifier.weight	MISSING
4. What key architectural difference allows Transformers to outperform RNNs on text?

Your answers:

- UNEXPECTED :can be ignored when loading from different task/architecture;

1. In terms of accuracy and overall performance, DistilBERT outperforms LSTM and GRU. Pre-training on large datasets allows the model to understand language patterns, grammar, and context before fine-tuning, rather than starting from scratch.

Training config: {'batch_size': 16, 'lr': 2e-05, 'epochs': 3}

2. Compared to LSTM and GRU, DistilBERT is slower to train per epoch. This is because it has more parameters and uses complex computational mechanisms, which require more computation time.
3. DistilBERT only needs 3 epochs to complete training because it is pre-trained, and fine-tuning converges quickly. RNNs, on the other hand, require 5 epochs because they learn from scratch. If DistilBERT is trained for 10 epochs, it may overfit the training data, meaning its performance on new data may degrade.
4. Transformers use self-attention, enabling them to process all words in a sentence simultaneously. RNNs, on the other hand, process text word by word, making them less adept at capturing long-range dependencies.

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture;

- MISSING :those params were newly initialized because missing from the check

Epoch [1/3] Loss: 0.4084

Epoch [2/3] Loss: 0.1398

Epoch [3/3] Loss: 0.1398

Part C: Vision Transformers (ViTs)

The Transformer architecture is not limited to language. In 2020, Google showed that

the same attention mechanism works remarkably well on images.

The key idea is to cut an image into small square patches, treat each patch as a "token,"

and feed them into a standard Transformer encoder. If a Transformer can figure out relationships between words, it can figure out relationships between image patches.

You will:	vocab_transformer.bias	UNEXPECTED
	vocab_projector.bias	UNEXPECTED
1. Load a pre-trained ViT model	vocab_transform.weight	UNEXPECTED
	vocab_layer_norm.bias	UNEXPECTED
2. Fine-tune on CIFAR-10 (same data set from Module 03)	classifier.bias	MISSING
3. Visualize attention maps – what does the model focus on?	classifier.weight	MISSING
4. Compare to your ViT from Module 03 results		MISSING

Notes:

```
# Cell C.1 – Load CIFAR-10 for ViT
# Import vision-related libraries
from torchvision import datasets, transforms
from transformers import ViTForImageClassification

# CIFAR-10 class names
CIFAR_CLASSES = ['airplane', 'automobile', 'bird', 'cat', 'deer',
                 'dog', 'frog', 'horse', 'ship', 'truck']

# Define image transformations for ViT
# ViT expects 224x224 images (CIFAR-10 is originally 32x32)
transform_vit = transforms.Compose([
    transforms.Resize((224, 224)), # Resize to ViT input size
    transforms.ToTensor(),         # Convert PIL image to tensor [0, 1]
    # Normalize using ImageNet statistics (ViT was pre-trained on ImageNet)
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Load CIFAR-10 dataset
cifar_train_full = datasets.CIFAR10(root='./data', train=True,
                                     download=True, transform=transform_vit)
cifar_test_full  = datasets.CIFAR10(root='./data', train=False,
                                     download=True, transform=transform_vit)

# Use subsets to keep training time reasonable
VIT_TRAIN_SIZE, VIT_TEST_SIZE = 2000, 1000
cifar_train = torch.utils.data.Subset(cifar_train_full, range(VIT_TRAIN_SIZE))
cifar_test  = torch.utils.data.Subset(cifar_test_full, range(VIT_TEST_SIZE))

# Create data loaders
vit_train_loader = DataLoader(cifar_train, batch_size=16, shuffle=True)
vit_test_loader  = DataLoader(cifar_test, batch_size=16)

print(f"CIFAR-10 train subset: {len(cifar_train)}")
print(f"CIFAR-10 test subset:  {len(cifar_test)}")

# Visualize sample images
fig, axes = plt.subplots(1, 8, figsize=(16, 2))
for i, ax in enumerate(axes):
    img, label = cifar_train_full[i]
    # Denormalize for display
    img_display = np.clip(img.permute(1, 2, 0).numpy()*0.225+0.45, 0, 1)
```

```
ax.imshow(img_display)
ax.set_title(CIFAR_CLASSES[label], fontsize=9)
ax.axis('off')

plt.tight_layout()
plt.show()
```

100%|██████████| 170M/170M [00:00<00:00, 508.52kB/s] Materializing param=distilbert.transformer.encoder.layer.0.attention.self.key.weight

CIFAR-10 train subset: 2000

DistilBERT Test Subset Classification LOAD REPORT from: distilbert-base-uncased



vocab_projector.bias	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
pre_classifier.bias	MISSING
classifier.bias	MISSING
classifier.weight	MISSING

```
# Cell C.2 – Load Pre-trained ViT
# Import vision-related libraries
from transformers import ViTForImageClassification, ViTConfig

# Load the configuration first
config = ViTConfig.from_pretrained('google/vit-base-patch16-224')

# Modify the configuration for our task
config.num_labels = 10
config.output_attentions = True # Explicitly enable attention output

# Load Vision Transformer model with the modified configuration
# - 'google/vit-base-patch16-224': base model, 16x16 patches, 224x224 input
# - num_labels=10: modify output layer for CIFAR-10 (10 classes)
# - ignore_mismatched_sizes=True: allow output layer size mismatch
vit_model = ViTForImageClassification.from_pretrained(
    'google/vit-base-patch16-224',
    config=config,
    ignore_mismatched_sizes=True).to(device)

print(f"ViT Parameters: {sum(p.numel() for p in vit_model.parameters()):,}")
print(f"\nArchitecture:")
print(f" Patch size: 16x16 | Image: 224x224 | Patches: {(224//16)**2} ({224//16}x{224//16})")
print(f" Hidden: 768 | Heads: 12 | Layers: 12")
print(f"\nHow it works:")
print(f" 1. Split 224x224 image into 196 patches (14x14 grid of 16x16 patches)")
print(f" 2. Flatten each patch into a 768-dimensional vector")
print(f" 3. Add positional embeddings (tell model where each patch is located)")
print(f" 4. Process through 12 Transformer layers with self-attention")
print(f" 5. Use [CLS] token output for classification")

=== BERT Hyperparameter Results ===
batch_size learning_rate epochs accuracy training_time_sec
1 1e-05 100000 0.88888 273.551265
```

```

1         16         0.00002         5         91.20         273.554585
2 Loading weights: 100% 200/200 [00:00<00:00, 597.52kB/s, Materializing param=Vit.layerNorm.weight]
3         16         0.00001         3         90.60         273.396224
4 ViTForImageClassification LOAD REPORT from google/vit-base-patch16-224
5 Key         16         | Stat 0.00002 |         2         90.15         182.319520
6 -----16-----+-----3-----89.80-----273.542312-----
7 classifier.weight | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([100
8 classifier.bias   | MISMATCH | Reinit due to size mismatch ckpt: torch.Size([100

=== BERT vs RNN Training Characteristics ===
Notes:
1. MISMATCH: ckpt weights were loaded, but they did not match the original empty
   ViT parameters.
2. Optimal Hyperparameters: BERT typically requires very small learning rates (1e-5 to 5e-5),
   while RNNs can tolerate much larger learning rates (e.g., 1e-3).
Architecture:
2. Patch size: 16x16 | Image: 224x224 | Patches: 196 (14x14 grid)
   Hidden: 768 | Heads: 12 | Layers: 12
   BERT is significantly more sensitive to learning rate changes.
   A slightly high LR (e.g., 5e-5) may destabilize fine-tuning.
How it works: RNNs are generally more stable across LR variations.
1. Split 224x224 image into 196 patches (14x14 grid of 16x16 patches)
2. Flatten each patch into a 768-dimensional vector
3. Add positional embeddings (tell model where each patch is located)
4. Process through 12 Transformer layers with self-attention
5. Use [CLS] token output for classification

```

Knowledge Check C.1

1. **How does ViT convert an image into a sequence?** Describe the patch embedding process.
2. **Why is positional encoding important for ViT?** What would happen without it?
3. **How is ViT fundamentally different from the CNNs you built in Module 03?** (Local vs. global processing)

Your answers:

1. The ViT segments an image into fixed-size image patches. Each patch is flattened into a one-dimensional vector, and then image patch embeddings are generated through a linear projection layer. These embeddings form a sequence, which is then fed into the Transformer encoder.
2. Positional encoding is important because the model itself cannot understand the spatial relationships between image patches. Without positional encoding, the model would treat image patches as disordered fragments, thus losing spatial structure and reducing model accuracy.
3. ViT can perform global image processing, meaning that each image patch can focus on all other image patches from the beginning. However, CNN first needs to

learn local features and then gradually build a global understanding through deeper layers.

```
# Cell C.3 – Fine-tune ViT

# Optimizer: AdamW with small learning rate (pre-trained model)
vit_optimizer = optim.AdamW(vit_model.parameters(), lr=2e-5, weight_decay=0.01)

VIT_EPOCHS = 3
vit_history = {'train_loss':[], 'train_acc':[], 'test_acc':[], 'epoch_times':[]}

print("=" * 60)
print("Fine-tuning Vision Transformer on CIFAR-10")
print("=" * 60)

for epoch in range(VIT_EPOCHS):
    # ===== TRAINING PHASE =====
    vit_model.train()
    total_loss = correct = total = 0
    start = time.time()

    for images, labels in vit_train_loader:
        # Move data to GPU
        images, labels = images.to(device), labels.to(device)

        # Zero gradients
        vit_optimizer.zero_grad()

        # Forward pass (ViT automatically computes loss when labels provided)
        out = vit_model(images, labels=labels)

        # Backward pass
        out.loss.backward()

        # Gradient clipping
        torch.nn.utils.clip_grad_norm_(vit_model.parameters(), 1.0)

        # Update weights
        vit_optimizer.step()

        # Track metrics
        total_loss += out.loss.item()
        _, pred = out.logits.max(1)
        correct += (pred == labels).sum().item()
        total += labels.size(0)

    et = time.time() - start
    ta = correct/total

    # ===== EVALUATION PHASE =====
```

```

vit_model.eval()
tc = tt = 0

with torch.no_grad():
    for images, labels in vit_test_loader:
        images, labels = images.to(device), labels.to(device)
        _, pred = vit_model(images).logits.max(1)
        tc += (pred == labels).sum().item()
        tt += labels.size(0)

tea = tc/tt

# Record metrics
vit_history['train_loss'].append(total_loss/len(vit_train_loader))
vit_history['train_acc'].append(ta)
vit_history['test_acc'].append(tea)
vit_history['epoch_times'].append(et)

# Display progress
print(f" Epoch {epoch+1}/{VIT_EPOCHS} | Loss: {total_loss/len(vit_train_lc

```

```

=====
Fine-tuning Vision Transformer on CIFAR-10
=====
Epoch 1/3 | Loss: 1.0637 | Train: 0.7605 | Test: 0.9560 | 75.1s
Epoch 2/3 | Loss: 0.1003 | Train: 0.9825 | Test: 0.9740 | 73.9s
Epoch 3/3 | Loss: 0.0122 | Train: 0.9995 | Test: 0.9730 | 73.7s

```

Student Coding Exercise C.4 — Visualizing ViT Attention

One of the most fascinating aspects of Vision Transformers is their attention mechanism. Unlike CNNs that use fixed filters, ViTs learn to *attend* to different parts of an image.

Your task:

1. Extract attention weights from the ViT model
2. Visualize attention maps for 5 different images
3. Compare attention patterns across different classes
4. Identify what the model "focuses on" for each class

Expected output:

- 5 visualizations showing: original image + attention heatmap overlay
- Written analysis explaining what the model attends to

Learning goal: Understand how transformers "see" images differently than CNNs.

```

# Cell C.4 – ViT Attention Visualization
#  YOUR CODE HERE: Extract and visualize attention maps

```

```

import torch
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

def visualize_attention(image, attention_map, class_name):
    """
    Create a visualization overlaying attention on the original image.

    Args:
        image: Original image
        attention_map: 2D attention weights
        class_name: Class label for the image
    """
    # --- Denormalize image tensor (C,H,W) → numpy (H,W,3) in [0,1] ---
    # Ensure image is on CPU before converting to numpy
    img_np = image.cpu().permute(1, 2, 0).numpy()
    # CIFAR-10 is normalized with ImageNet stats, so we reverse it for display
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img_np = img_np * std + mean
    img_np = np.clip(img_np, 0, 1)
    H, W = img_np.shape[:2]

    # --- Upsample coarse attention grid → full image resolution ---
    attn_t = torch.tensor(attention_map).unsqueeze(0).unsqueeze(0) # (1,1,g,g)
    attn_up = F.interpolate(attn_t, size=(H, W), mode='bilinear', align_corners=False)
    attn_up = attn_up.squeeze().numpy() # (H, W)

    # --- Build alpha-blended overlay (jet colormap + original) ---
    colormap = cm.get_cmap('jet')
    attn_rgb = colormap(attn_up)[..., :3] # (H, W, 3)
    overlay = np.clip(0.60 * img_np + 0.40 * attn_rgb, 0, 1)

    # --- Plot: original | attention heatmap | overlay ---
    fig, axes = plt.subplots(1, 3, figsize=(12, 4))
    fig.suptitle(f"Class: {class_name}", fontsize=13, fontweight='bold')

    axes[0].imshow(img_np)
    axes[0].set_title("Original Image")
    axes[0].axis('off')

    axes[1].imshow(attn_up, cmap='hot', vmin=0, vmax=1)
    axes[1].set_title("Attention Map\n(bright = high attention)")
    axes[1].axis('off')

    axes[2].imshow(overlay)
    axes[2].set_title("Overlay\n(attention on image)")
    axes[2].axis('off')

```

```

plt.tight_layout()
plt.show()

# Select 5 images from different classes
# Scan the test loader for one sample per target class
TARGET_CLASSES = {0: 'airplane', 1: 'automobile', 2: 'bird', 3: 'cat', 4: 'deer'}
selected_images = {} # { class_idx : image_tensor }
selected_labels = {} # { class_idx : class_name }

vit_model.eval()
for imgs, lbls in vit_test_loader:
    for img, lbl in zip(imgs, lbls):
        c = lbl.item()
        if c in TARGET_CLASSES and c not in selected_images:
            selected_images[c] = img
            selected_labels[c] = TARGET_CLASSES[c]
            if len(selected_images) == len(TARGET_CLASSES):
                break
    if len(selected_images) == len(TARGET_CLASSES):
        break

print(f" Collected images for: {list(selected_labels.values())}")
for class_idx in sorted(selected_images.keys()):
    img = selected_images[class_idx]
    class_name = selected_labels[class_idx]

    # --- Attention extraction logic (integrated) ---
    with torch.no_grad():
        outputs = vit_model(img.unsqueeze(0).to(device), output_attentions=True)
        attentions = outputs.attentions # Attention weights from all layers

    last_attn = attentions[-1].cpu().mean(dim=1).squeeze(0) # (seq_len, seq_len)
    cls_attn = last_attn[0, 1:]
    num_patches = cls_attn.shape[0]
    grid_size = int(np.sqrt(num_patches))
    attention_map = cls_attn.reshape(grid_size, grid_size).numpy()

    attention_map = (attention_map - attention_map.min()) / \
        (attention_map.max() - attention_map.min() + 1e-8)
    # --- End Attention extraction logic ---

    # Visualize
    visualize_attention(img, attention_map, class_name)

    # Optional: Save each figure individually
    # fig_path = f"attention_{class_name}.png"
    # plt.savefig(fig_path, dpi=120, bbox_inches='tight')
    # print(f" Saved → {fig_path}")

# Analysis

```

```
print("\n=== Attention Pattern Analysis ===\n1. Airplanes: The model tends to f
```


Collected images for: ['cat', 'airplane', 'automobile', 'deer', 'bird']

✓ C.5 — Visualize ViT Attention Maps

Attention maps show which image regions the model focuses on

`vit_model.eval()`

Get a batch of test images `sample_images, sample_labels = next(iter(vit_test_loader))`

`sample_images = sample_images.to(device)`

Forward pass with attention output enabled

with `torch.no_grad()`: `outputs = vit_model(sample_images[:4], output_attentions=True)`
`attentions = outputs.attentions` # Attention weights from all layers

Extract attention from the last layer

Average across all attention heads to get a single attention map

`last_attn = attentions[-1].cpu().mean(dim=1)` # Shape: (batch, seq_len, seq_len)

Extract attention from [CLS] token to all patch tokens

[CLS] token (index 0) is used for classification

Reshape to 14×14 grid (196 patches = 14×14)

`cls_attn = last_attn[:, 0, :].reshape(-1, 14, 14)`

Visualize images and attention maps

`fig, axes = plt.subplots(2, 4, figsize=(16, 8))` for `i in range(4)`: # Top row: original images
 with predictions `img = sample_images[i].cpu().permute(1,2,0).numpy()` # Denormalize for display
`img = np.clip(img * np.array([0.229, 0.224, 0.225]) + np.array([0.485, 0.456, 0.406]), 0, 1)` `axes[0,i].imshow(img)` `axes[0,i].axis('off')`

```
##### Get prediction and true label
pred = outputs.logits[i].argmax().item()
true = sample_labels[i].item()
color = 'green' if pred == true else 'red'
axes[0,i].set_title(f"True: {CIFAR_CLASSES[true]}\nPred: {CIFAR_CLASSES[pred]}",
                    fontsize=10, color=color)

# Bottom row: attention maps
```

```
# Bright regions = high attention (model focuses here)
axes[1,i].imshow(cls_attn[i].numpy(), cmap='hot', interpolation='bilinear')
axes[1,i].set_title('Attention Map', fontsize=10)
axes[1,i].axis('off')
```

```
plt.suptitle("Vision Transformer: What Does the Model Focus On?", fontweight='bold',
            fontsize=14) plt.tight_layout() plt.show()
== Attention Pattern Analysis ==
```

```
1. Airplanes: The model tends to focus on wings and fuselage regions,
   indicating attention to global shape structure.
2. Cars: Attention often highlights the body and wheel areas.
```

Start coding or [generate](#) with AI.

though backgrounds can sometimes receive partial attention.

5. Unlike CNN feature maps, ViT attention is global rather than local,



5. This suggests ViTs 'understand' images by modeling patch-to-patch

Student Experimentation C.5 – ViT vs CNN Performance Comparison

6. This suggests ViTs 'understand' images by modeling patch-to-patch relationships instead of purely hierarchical convolution features.

In Module 03, you built CNNs for image classification. Now let's directly compare CNN and ViT performance on the same task.

Your task:

1. Load a CNN model (either from Module 03 or create a simple one)
2. Train it on the same CIFAR-10 subset used for ViT
3. Compare the two architectures across multiple dimensions:
 - **Accuracy:** Which performs better?
 - **Training time:** Which is faster to train?
 - **Parameters:** Which is more parameter-efficient?
 - **Inference speed:** Which is faster at test time?
4. Write a comprehensive analysis of when to use each architecture

Expected output:

- Comparison table with all metrics
- 5-7 sentence analysis with practical recommendations

```
# Cell C.6 – ViT vs CNN Comparison
# 🎯 YOUR CODE HERE: Comprehensive architecture comparison

# TODO: Define a simple CNN for comparison
class SimpleCNN(nn.Module):
    """
    Simple CNN for CIFAR-10 classification.

    Architecture: Conv layers → Pooling → Fully connected
```

```

"""
def __init__(self, num_classes=10):
    super().__init__()
    self.features = nn.Sequential(
        # Block 1 – 3 → 32 channels, 224→112
        nn.Conv2d(3, 32, kernel_size=3, padding=1),
        nn.BatchNorm2d(32),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2, 2),
        # Block 2 – 32 → 64 channels, 112→56
        nn.Conv2d(32, 64, kernel_size=3, padding=1),
        nn.BatchNorm2d(64),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2, 2),
        # Block 3 – 64 → 128 channels, 56→28
        nn.Conv2d(64, 128, kernel_size=3, padding=1),
        nn.BatchNorm2d(128),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2, 2),
    )
    self.classifier = nn.Sequential(
        nn.Flatten(),
        nn.Linear(128 * 28 * 28, 256),
        nn.ReLU(inplace=True),
        nn.Dropout(0.4),
        nn.Linear(256, num_classes),
    )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x

# Train the CNN
print("=== Training CNN ===")
cnn_model = SimpleCNN(num_classes=10).to(device)
cnn_optimizer = optim.Adam(cnn_model.parameters(), lr=1e-3, weight_decay=1e-4)
cnn_criterion = nn.CrossEntropyLoss()
CNN_EPOCHS = VIT_EPOCHS # same number of epochs as ViT for a fair
cnn_total_time = 0.0
for epoch in range(CNN_EPOCHS):
    cnn_model.train()
    total_loss = correct = total = 0
    start = time.time()

    for images, labels in vit_train_loader:
        images, labels = images.to(device), labels.to(device)
        cnn_optimizer.zero_grad()
        logits = cnn_model(images)
        loss = cnn_criterion(logits, labels)

```

```

        loss.backward()
        cnn_optimizer.step()

    total_loss += loss.item()
    _, pred = logits.max(1)
    correct += (pred == labels).sum().item()
    total += labels.size(0)

et = time.time() - start
cnn_total_time += et

cnn_model.eval()
tc = tt = 0
with torch.no_grad():
    for images, labels in vit_test_loader:
        images, labels = images.to(device), labels.to(device)
        _, pred = cnn_model(images).max(1)
        tc += (pred == labels).sum().item()
        tt += labels.size(0)

    print(f" Epoch {epoch+1}/{CNN_EPOCHS} | Loss: {total_loss/len(vit_train_loader)} | Train: {correct/total:.4f} | Test: {tc/tt:.4f} | {et:.1f}s")

cnn_accuracy = tc / tt
cnn_train_time = cnn_total_time
cnn_params = sum(p.numel() for p in cnn_model.parameters() if p.requires_grad)

# Measure inference speed
print("\n=== Measuring Inference Speed ===")
sample_imgs, _ = next(iter(vit_test_loader))
sample_imgs = sample_imgs[:100].to(device)
RUNS = 20 # average over multiple runs for stable timing
cnn_model.eval()
vit_model.eval()

# warm-up passes (avoids cold-start GPU overhead)
with torch.no_grad():
    _ = cnn_model(sample_imgs)
    _ = vit_model(sample_imgs)

# CNN timing
with torch.no_grad():
    if device.type == 'cuda': torch.cuda.synchronize()
    t0 = time.time()
    for _ in range(RUNS):
        _ = cnn_model(sample_imgs)
    if device.type == 'cuda': torch.cuda.synchronize()
    cnn_inference_time = (time.time() - t0) / RUNS * 1000 # ms per 100 images

# ViT timing
with torch.no_grad():

```

```

    if device.type == 'cuda': torch.cuda.synchronize()
    t0 = time.time()
    for _ in range(RUNS):
        _ = vit_model(sample_imgs)
    if device.type == 'cuda': torch.cuda.synchronize()
    vit_inference_time = (time.time() - t0) / RUNS * 1000    # ms per 100 images

print(f"  CNN inference (100 imgs): {cnn_inference_time:.2f} ms")
print(f"  ViT inference (100 imgs): {vit_inference_time:.2f} ms")

# Get ViT metrics (from C.3 or retrain)
vit_accuracy  = vit_history['test_acc'][-1]
vit_train_time = sum(vit_history['epoch_times'])
vit_params    = sum(p.numel() for p in vit_model.parameters() if p.requires_grad)

# Create comprehensive comparison table
print("\n=== ViT vs CNN Comparison ===")
print(f"{'Metric':<20} {'CNN':<15} {'ViT':<15} {'Winner':<10}")
print("-" * 60)
def _winner(cnn_val, vit_val, higher_is_better=True):
    if higher_is_better:
        return "CNN" if cnn_val >= vit_val else "ViT"
    else:
        return "CNN" if cnn_val <= vit_val else "ViT"

rows = [
    ("Test Accuracy",
     f"{cnn_accuracy*100:.2f}%",
     f"{vit_accuracy*100:.2f}%",
     _winner(cnn_accuracy, vit_accuracy, higher_is_better=True)),

    ("Train Time (s)",
     f"{cnn_train_time:.1f}s",
     f"{vit_train_time:.1f}s",
     _winner(cnn_train_time, vit_train_time, higher_is_better=False)),

    ("Parameters",
     f"{cnn_params:,}",
     f"{vit_params:,}",
     _winner(cnn_params, vit_params, higher_is_better=False)),

    ("Inference (ms/100)",
     f"{cnn_inference_time:.2f}ms",
     f"{vit_inference_time:.2f}ms",
     _winner(cnn_inference_time, vit_inference_time, higher_is_better=False)),

    ("Pre-training needed", "No", "Yes (ImageNet)", "CNN"),
    ("Global context", "Limited", "Yes (attention)", "ViT"),
    ("Small-data perf.", "Strong", "Needs more data", "CNN"),
]

```

```

for metric, cnn_val, vit_val, w in rows:
    print(f" {metric:<18} {cnn_val:<15} {vit_val:<15} {w:<10}")

# Bar chart summary
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle("ViT vs CNN – Head-to-Head", fontsize=14, fontweight='bold')
colors = ['#2E86C1', '#E67E22']

axes[0].bar(['CNN', 'ViT'], [cnn_accuracy*100, vit_accuracy*100], color=colors)
axes[0].set_title("Test Accuracy (%)")
axes[0].set_ylim(0, 100)
for i, v in enumerate([cnn_accuracy*100, vit_accuracy*100]):
    axes[0].text(i, v+1, f"{v:.1f}%", ha='center', fontweight='bold')

axes[1].bar(['CNN', 'ViT'], [cnn_train_time, vit_train_time], color=colors)
axes[1].set_title("Total Train Time (s)")
for i, v in enumerate([cnn_train_time, vit_train_time]):
    axes[1].text(i, v+1, f"{v:.0f}s", ha='center', fontweight='bold')

axes[2].bar(['CNN', 'ViT'], [cnn_params/1e6, vit_params/1e6], color=colors)
axes[2].set_title("Parameters (M)")
for i, v in enumerate([cnn_params/1e6, vit_params/1e6]):
    axes[2].text(i, v+0.2, f"{v:.1f}M", ha='center', fontweight='bold')

plt.tight_layout()
plt.show()

# Comprehensive analysis
print("\n=== Architecture Comparison Analysis ===")
print("1. Which architecture performed better overall?")
print(f" ViT achieved {vit_accuracy*100:.1f}% vs CNN's {cnn_accuracy*100:.1f}%  

    f"entirely from ImageNet pre-training – it arrived with rich visual featur")
print("2. What are the trade-offs between them?")
print(f" ViT has {vit_params/cnn_params:.0f}x more parameters and took {vit_train_time:.0f}s  

    f"longer to train; the CNN is leaner, faster, and needs no pre-training p")
print("3. When would you choose CNN over ViT?")
print(" When data is limited (<10k images), hardware is constrained, or input  

    "– CNNs' built-in local/translation inductive biases make them data-efficient")
print("4. When would you choose ViT over CNN?")
print(" When a pre-trained checkpoint is available, the dataset is large (100k+)  

    "or global context matters (e.g. scene understanding, medical imaging).")
print("5. How does dataset size affect this decision?")
print(" ViTs underperform CNNs when trained from scratch on small datasets because of  

    "spatial inductive biases; pre-training on a large corpus is practically necessary")
print("6. What about computational resources?")
print(f" CNN inference ({cnn_inference_time:.1f}ms) is faster than ViT ({vit_inference_time:.1f}ms)  

    "ViT's  $O(n^2)$  attention also consumes more GPU memory, making CNNs the preferred choice  

    "for edge deployment or real-time applications.")

```


=== Training CNN ===



Epoch 1/3 | Loss: 5.6033 | Train: 0.1325 | Test: 0.1410 | 8.1s
Epoch 2/3 | Loss: 2.2548 | Train: 0.1315 | Test: 0.1960 | 7.9s
Epoch 3/3 | Loss: 2.1829 | Train: 0.1645 | Test: 0.2030 | 8.2s

1. **Look at the attention maps.** Do bright regions correspond to the main objects?

=== Measuring Inference Speed ===

What does this tell you about how ViT classifies?

CNN Inference (100 imgs): 11.82 ms

2. **How does ViT's accuracy compare to the CNN you built in Module 03?** (Typical

CNN on CIFAR-10: 75–85%)

=== ViT vs CNN Comparison ===

3. **ViT has far more parameters than a CNN. Is that always an advantage?** When

might a CNN be better?

Test Accuracy	20.30%	97.30%	ViT
Train Time (s)	24.2s	222.6s	CNN
Parameters	25,786,634	85,806,346	CNN
Inference (ms/100)	11.82ms	206.29ms	CNN

Your answers:

1. Yes, bright regions in the attention map typically correspond to the main object in the image. This indicates that ViT is able to identify key visual features during classification and utilizes an attention mechanism to weight the most relevant regions for the final prediction.

2. With proper training or pre-training, ViT can achieve slightly higher accuracy than CNNs. This demonstrates that Transformer models can be improved with large datasets and pre-training.

3. Having more parameters isn't always an advantage; it depends on the task. CNNs may perform better when the dataset is small and computational resources are limited.

=== Architecture Comparison Analysis ===

1. Which architecture performed better overall?

ViT achieved 97.3% vs CNN's 20.3%, but ViT's edge comes entirely from ImageNet

2. What are the trade-offs between them?

ViT has 3x more parameters and took 9.2x longer to train; the CNN is leaner,

3. When would you choose CNN over ViT?

When data is limited (<10k images), hardware is constrained, or inputs are low

4. When would you choose ViT over CNN?

When a pre-trained checkpoint is available, the dataset is large (100k+ image

5. How does dataset size affect this decision?



Student Exercise D1—Create Master Comparison Table

6. What about computational resources?

CNN inference (11.8ms) is 19x faster than ViT (206.2ms) per 100 images. ViT's O(n

You've now trained and evaluated 7 different models across text and image tasks.

- **Part A:** Vanilla RNN, LSTM, GRU (text classification)
- **Part B:** DistilBERT (text classification)
- **Part C:** CNN, ViT (image classification)

Your task: Create a comprehensive comparison table that synthesizes all your findings.

Required columns:

- Model name

- Task (text or image)
- Test accuracy
- Training time
- Number of parameters
- Key strengths
- Key weaknesses
- Best use cases

```
# Cell D.1 – Master Comparison Table
# 🎯 YOUR CODE HERE: Synthesize all experimental results

import pandas as pd
# Compile all results from Parts A, B, and C
# Create a comprehensive comparison table

# Example structure:
results_summary = {
    'Vanilla RNN': {
        'task': 'Text Classification',
        'accuracy': vanilla_history['test_acc'][-1],
        'train_time': sum(vanilla_history['epoch_times']),
        'parameters': sum(p.numel() for p in vanilla_model.parameters() if p.requires_grad_()),
        'strengths': 'Simple, fast to train, low memory',
        'weaknesses': 'Vanishing gradients, no long-range memory',
        'use_cases': 'Short sequences, real-time edge inference'
    },
    'LSTM': {
        'task': 'Text Classification',
        'accuracy': lstm_history['test_acc'][-1],
        'train_time': sum(lstm_history['epoch_times']),
        'parameters': sum(p.numel() for p in lstm_model.parameters() if p.requires_grad_()),
        'strengths': 'Handles long-range dependencies, gated memory',
        'weaknesses': 'Sequential – cannot parallelise across time steps',
        'use_cases': 'Time series, speech, moderate-length text'
    },
    'GRU': {
        'task': 'Text Classification',
        'accuracy': gru_history['test_acc'][-1],
        'train_time': sum(gru_history['epoch_times']),
        'parameters': sum(p.numel() for p in gru_model.parameters() if p.requires_grad_()),
        'strengths': 'Fewer params than LSTM, similar accuracy, faster',
        'weaknesses': 'Still sequential, weaker on very long sequences',
        'use_cases': 'Resource-constrained NLP, streaming text'
    },
    'DistilBERT': {
        'task': 'Text Classification',
        'accuracy': bert_history['test_acc'][-1],
        'train_time': sum(bert_history['epoch_times']),

```

```

        'parameters': sum(p.numel() for p in bert_model.parameters() if p.requires_grad_()),
        'strengths': 'Bidirectional context, pre-trained, state-of-the-art NLP',
        'weaknesses': 'Large model, slow inference, needs GPU',
        'use_cases': 'Production NLP, sentiment, QA, summarisation'
    },
    'SimpleCNN': {
        'task': 'Image Classification',
        'accuracy': cnn_accuracy,
        'train_time': cnn_train_time,
        'parameters': cnn_params,
        'strengths': 'Fast, data-efficient, low memory, no pre-training',
        'weaknesses': 'Limited global context, hand-crafted inductive biases',
        'use_cases': 'Edge vision, small datasets, real-time detection'
    },
    'ViT': {
        'task': 'Image Classification',
        'accuracy': vit_accuracy,
        'train_time': vit_train_time,
        'parameters': vit_params,
        'strengths': 'Global attention, highly scalable, interpretable maps',
        'weaknesses': 'Needs large data / pre-training, slow, memory-heavy',
        'use_cases': 'Large-scale vision, medical imaging, scene understanding'
    }
}

# Create a well-formatted table

df = pd.DataFrame(results_summary).T.reset_index()
df.columns = ['Model', 'Task', 'Accuracy', 'Train Time (s)',
              'Parameters', 'Strengths', 'Weaknesses', 'Use Cases']

# Round numeric columns for readability
df['Accuracy'] = df['Accuracy'].astype(float).map(lambda x: f"{x*100:.2f}%")
df['Train Time (s)'] = df['Train Time (s)'].astype(float).map(lambda x: f"{x:.1f}")
df['Parameters'] = df['Parameters'].astype(int).map(lambda x: f"{x:,}")

print("=" * 80)
print("MASTER COMPARISON TABLE – All Models")
print("=" * 80)
print(df.to_string(index=False))
print()

# Styled HTML table (renders inline in Colab / Jupyter)
styled = (
    df.style
        .set_caption("Master Model Comparison – ITAI 2376 Lab 05")
        .set_table_styles([
            {'selector': 'caption',
             'props': [('font-size', '14px'), ('font-weight', 'bold'), ('padding', '10px 0 0 0)]},
            {'selector': 'th',
             'props': [('background-color', '#2E4057'), ('color', 'white'), ('padding', '5px 10px 5px 10px)]}])
)

```

```

        ('text-align', 'center'), ('padding', '6px')]],
        {'selector': 'tr:nth-child(even)',
         'props': [('background-color', '#f2f2f2')]}],
    ])
    .hide(axis='index')
)
display(styled)

# Create visualizations
# 1. Bar chart comparing accuracies
# 2. Bar chart comparing training times
# 3. Scatter plot: accuracy vs parameters

# Pull raw numeric values back out for plotting
model_names = list(results_summary.keys())
accuracies = [results_summary[m]['accuracy'] * 100 for m in model_names]
train_times = [results_summary[m]['train_time'] for m in model_names]
param_counts = [results_summary[m]['parameters'] / 1e6 for m in model_names]
tasks = [results_summary[m]['task'] for m in model_names]

# Colour by task so text vs image models stand out
task_colors = ['#2E86C1' if t == 'Text Classification' else '#E67E22' for t in tasks]

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle("Master Comparison – All 6 Models", fontsize=15, fontweight='bold')

# — 1. Bar chart: Accuracy —————
bars1 = axes[0].bar(model_names, accuracies, color=task_colors, edgecolor='white')
axes[0].set_title("Test Accuracy (%)", fontsize=12, fontweight='bold')
axes[0].set_ylabel("Accuracy (%)")
axes[0].set_ylim(0, 105)
axes[0].set_xticklabels(model_names, rotation=30, ha='right', fontsize=9)
for bar, val in zip(bars1, accuracies):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.8,
                  f"{val:.1f}%", ha='center', va='bottom', fontsize=8, fontweight='normal')

# — 2. Bar chart: Training Time —————
bars2 = axes[1].bar(model_names, train_times, color=task_colors, edgecolor='white')
axes[1].set_title("Total Training Time (s)", fontsize=12, fontweight='bold')
axes[1].set_ylabel("Seconds")
axes[1].set_xticklabels(model_names, rotation=30, ha='right', fontsize=9)
for bar, val in zip(bars2, train_times):
    axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                  f"{val:.0f}s", ha='center', va='bottom', fontsize=8, fontweight='normal')

# — 3. Scatter plot: Accuracy vs Parameters —————
for name, acc, params, color in zip(model_names, accuracies, param_counts, task_colors):
    axes[2].scatter(params, acc, color=color, s=120, zorder=3)
    axes[2].annotate(name, (params, acc),
                     textcoords="offset points", xytext=(6, 4), fontsize=8)
axes[2].set_title("Accuracy vs Parameters", fontsize=12, fontweight='bold')

```

```
axes[2].set_xlabel("Parameters (Millions)")
axes[2].set_ylabel("Test Accuracy (%)")
axes[2].set_ylim(0, 105)
axes[2].grid(True, linestyle='--', alpha=0.5)

# Shared legend for task colours
from matplotlib.patches import Patch
legend_handles = [Patch(color='#2E86C1', label='Text Classification'),
                  Patch(color='#E67E22', label='Image Classification')]
fig.legend(handles=legend_handles, loc='lower center', ncol=2,
           fontsize=10, bbox_to_anchor=(0.5, -0.04))

plt.tight_layout()
plt.savefig("master_comparison.png", dpi=150, bbox_inches='tight')
plt.show()
print(" Figure saved → master_comparison.png")
```


MASTER COMPARISON TABLE — All Models



Instructions

Answer the following questions based on **your actual experience** completing this lab. Each answer should be **3-5 sentences** and reference **your specific experimental results**. Generic or theoretical answers will not receive full credit.

Master Model Comparison — ITAI 2376 Lab 05

Question 1: The Surprise Factor

What was the **most surprising result** you encountered in this lab? Describe what you expected to happen versus what actually happened, and explain why you think there was